

Chapter 1: Software Engineering as Practical Problem Solving

Gregory S. DeLozier, PhD

Someone asks you to build a website. They have a few ideas about what it should look like, and you can already imagine how to write it. Getting started would be easy.

But what is the website supposed to improve?

That question can save a surprising amount of work. It can also be harder to answer than a question about the code. People usually describe the situation from where they stand. They may have a clear picture of something they want built without having worked out why the current arrangement keeps failing.

Before choosing a solution, we need to understand enough of the situation to make a useful choice. We can begin with an ordinary conversation and a close look at what people do now. This is a practical instance of problem solving (1).

What is actually going wrong?

Consider a campus club that runs a table at several events each semester. Members volunteer for one-hour shifts. The organizer posts a signup message in the club's group chat, and people reply with the times they can work.

At the last event, nobody arrived for the second shift. The organizer stayed for an extra hour and missed another commitment. A club member suggests building a volunteer website.

That sounds reasonable. A website could show a schedule and let people choose a shift. But we do not yet know why the shift was empty.

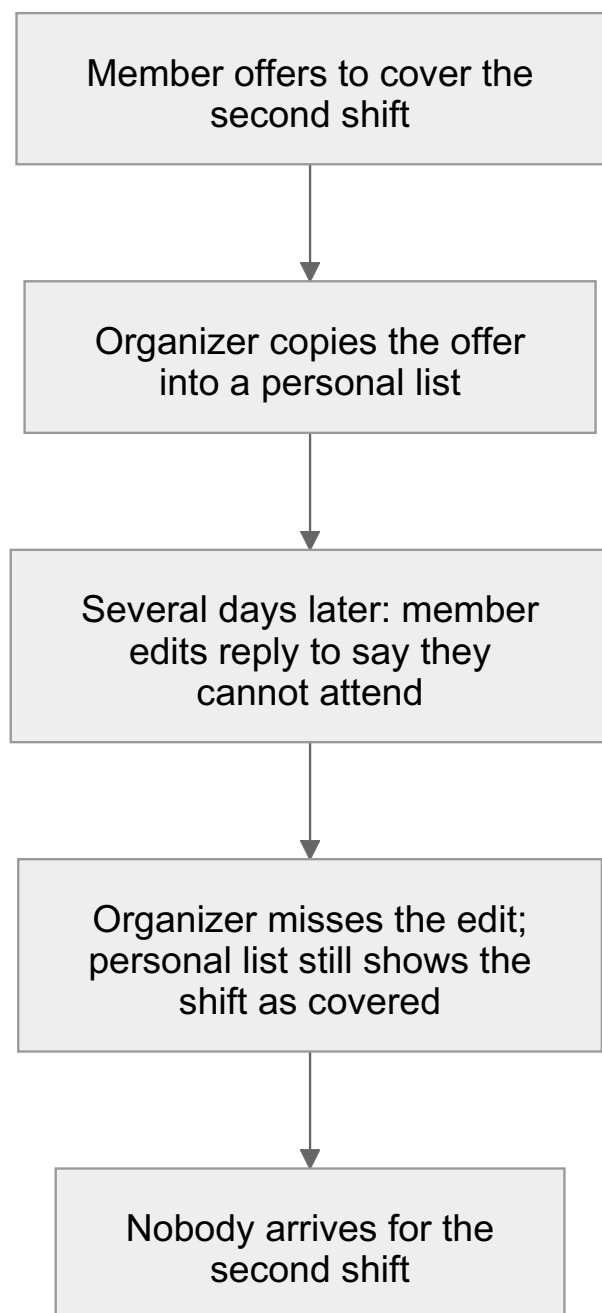
Perhaps nobody signed up for it. Perhaps someone signed up and later canceled. Perhaps two people thought the other person had agreed to cover it. Those are different problems, even though they produce the same empty table.

A useful first question is, "What happened last time?"

The organizer opens the chat. One member had offered to take the shift, then changed their reply several days later. The organizer had copied the original reply into a personal list and never saw the change. Meanwhile, another member had asked whether that shift was still available. Nobody answered.

Now we have something more specific than "signups are a mess." The people involved were looking at different accounts of who would be there. A change in one place did not reach the other people who needed to know.

Here is where the two accounts separated:



The personal list recorded an offer, but the organizer treated it as a confirmed commitment. When the reply changed, even that record of the offer became out of date. Making the list easier to read would not resolve either misunderstanding.

Notice how much came from looking at one actual event. General descriptions tend to leave out the awkward details. A particular missed shift has a sequence of actions we can examine. We can ask what each person knew at the time and where they got that information.

Who has the problem?

The organizer has an obvious problem: keeping the table staffed without spending the entire week chasing people. Club members have a different experience. They need to know which shifts are available and whether their offer to help has been accepted.

Imagine asking a member, “Why didn’t you sign up?”

They might say, “I asked about the afternoon shift, but I never heard back. I figured somebody else took it.”

The organizer might remember the same exchange differently: “I thought they were just asking. I was waiting for them to say they could do it.”

Neither person has to be careless for this to happen. One thought the exchange was over; the other thought it was still waiting for an answer.

We need to understand what people are trying to accomplish. The organizer wants a dependable schedule. A volunteer wants to make a commitment they can keep, with enough certainty to arrange the rest of their day. A solution has to work for both of them.

It is easy to pay attention only to the person who requested the software. That person may know the situation well, but other people can still reveal something important. The member who stopped volunteering may understand a difficulty that the organizer has never noticed.

Useful questions stay close to experience. “Show me how you signed up last time” is often easier to answer than “What features should the system have?” It also gives us less room to imagine that everybody works the way we would.

What would make things better?

What would count as an improvement? A volunteer should be able to tell which shift they have agreed to work, and the organizer should see the same commitment. If someone cancels, the organizer should find out in time to respond.

That description helps us judge a proposed solution. A shared schedule might help. A clear confirmation might help too. Member profiles probably have little to do with the failure we just examined.

There are also practical signs of improvement. Does the organizer still have to compare chat replies with a separate list? Do volunteers still ask whether they are on the schedule? At the next event, does an unfilled shift become visible before the event begins?

We should be careful about what one event can tell us. A fully staffed table would be encouraging, but it would not establish that every problem had been solved. Maybe the organizer quietly called everyone the night before. If the new arrangement still depends on that extra work, we need to know.

The improvement belongs to the people doing the work. A solution that looks simpler to its developer may make their day more difficult. Requiring an organizer to enter the same information twice is extra work, even if both screens are easy to use.

What is getting in the way?

The next event is in ten days. The club has no budget for a paid service. Its members already use the group chat. The organizer suspects that some would be reluctant to create another account just to volunteer for an hour. We should ask a few members before treating that suspicion as a constraint.

These facts affect what would be useful now. A custom application that takes six weeks to finish will not help with the next event. An unfamiliar service may ask more of the members than they are willing to do.

A reasonable first attempt could be a shared schedule with one organizer responsible for updating it. Volunteers can continue asking for shifts in the chat. The organizer confirms each commitment and points to the same schedule each time.

That arrangement has limitations. The organizer still does some manual work. Changes can still be missed. But it may remove the conflicting lists soon enough to help with the next event, and it gives us a chance to learn where the remaining trouble is.

There may be good reasons to write software later. If the club runs many events, keeping the schedule current by hand could become too much work. We would then have a clearer idea of which work needs help.

Time spent operating a solution counts too. Somebody will have to update a website and help people who cannot use it. Before adding those responsibilities, we should ask who can take them on. A student graduating at the end of the semester cannot be the club's only plan for keeping its application running.

What are we assuming?

Even a small proposed change rests on things we believe about the situation. Some of those beliefs will turn out to be wrong.

Each belief suggests something we could check:

What we are assuming	A quick way to check
Members can open the shared schedule.	Ask two members to open it on their phones.
Members understand when a shift is confirmed.	Ask them to explain what they think they have agreed to.
A missed shift means someone forgot.	Ask the person what happened before choosing a remedy.
There is enough volunteer time to cover the event.	Compare available hours with the hours that need coverage.

In the incident we examined, the member changed their reply and the organizer missed the update. A different missed shift could have a different cause: a volunteer's class might run late, or they might not find the event. A reminder would not necessarily solve either problem. We need to investigate each incident rather than assume that the same symptom always has the same cause.

The last assumption may be the most important. Suppose eight one-hour shifts need coverage, but members can offer only four hours in total. A better signup tool cannot supply the missing four hours. The club might need to shorten the event or recruit more help. Learning that before writing the application would be useful.

We do not have to resolve every uncertainty before doing anything. That would be another way to get stuck. We need enough understanding to choose a sensible next step, especially when being wrong would make a lot of our work unnecessary.

It helps to say what we do not know in plain language: “We haven’t checked whether members can open this without signing in.” That gives somebody a question they can answer. “It should be fine” gives the team much less to work with.

What could we try?

For the next event, the club could try the shared schedule with a small group of volunteers. Each person chooses a shift through the existing chat. The organizer records it and sends back a confirmation. A volunteer who needs to cancel contacts the organizer directly.

For example, the shared schedule could look like this. These are sample details for the trial:

Club information table - October 8, Student Center entrance

Shift	Confirmed volunteer	Status
10:00-11:00 a.m.	Alex	Confirmed
11:00 a.m.-noon	-	Open
Noon-1:00 p.m.	Sam	Confirmed

A confirmation to Sam could say:

You are confirmed for the club information table on October 8, noon-1:00 p.m., at the Student Center entrance. The shared schedule now shows your name. If your plans change, contact me directly so I can update it and find someone else.

The organizer sends this after recording the commitment. An unanswered offer in the chat is still just an offer. Both people can check the same schedule, and the empty shift is visible. This remains a manual arrangement, so a late or missed update is still possible.

The trial serves a purpose similar to software prototyping: learning from an early version before committing to a finished product (2). Here we can learn without writing an application yet.

Before using it for the event, ask two members to try the arrangement. Can they find an available shift? After receiving confirmation, can they tell exactly when they are expected? What would they do if their plans changed?

Watching people try the arrangement is a simple form of usability testing (3). If someone cannot find the schedule link, explaining where it is helps that person, but it also reveals something about the arrangement. Other members may have the same difficulty when nobody is beside them to explain it.

After the event, compare the experience with what happened before. Perhaps the conflicting lists are gone, but cancellations still arrive too late. That is a narrower problem than the one we started with. We can consider what to change next.

Perhaps the shared schedule works well enough that a custom application would add little value. That is a useful result too. The goal was to help the club run its events.

A small attempt gives us something real to discuss. People can react to an experience they have had, and we can see where our explanation of the problem was incomplete. We can then spend the next bit of effort with a better reason for doing so.

Doing work you can enjoy

There is a personal reason to develop these habits. Building the wrong thing is frustrating, particularly when you worked hard on it. So is finishing a task and discovering that the person who asked for it expected something else entirely.

Imagine spending an evening finishing a signup page, then learning that the organizer expected to approve every request. Now the behavior has to change, and both people may feel that the other wasted their time. Repeated often enough, these misunderstandings turn into late evenings and resentment.

Talking through one actual signup before making a large commitment can uncover that expectation while it is still easy to discuss. Better tools would not settle the misunderstanding for us. The way we work together matters to whether the job is enjoyable.

It also changes the conversation when something fails. We can ask which belief was wrong or what we did not notice. That gives us somewhere to go. Blaming someone for not predicting everything leaves the problem much as it was.

Clearer understanding does not guarantee easy work. There will still be difficult code and competing needs. But it gives the effort a purpose that the people involved can explain. It is easier to take pride in a modest improvement that helps someone than in an elaborate feature nobody needs.

Over a career, you will often hear requests that arrive with the solution already attached: build a dashboard, add a notification, replace this application. Take the request seriously, and get curious about the experience behind it. Understanding why somebody wants the change is part of doing the work well.

Try it with a situation you know

Choose a recurring difficulty from a club, a job, or an ordinary part of your day. Pick something specific enough that you can describe the last time it happened.

Work through these questions:

1. What happened, and what went wrong?
2. Who had difficulty, and what were they trying to do?
3. What would an improvement look like for those people?
4. What limits what you can change right now?
5. What are you assuming that a conversation or observation could check?
6. What small change could you try, and how would you tell whether it helped?

Leave the choice of programming language open. You may eventually have a good reason to write a program. First, get a clear enough picture of the situation that you can explain why that program would be worth building.

References

1. Wikipedia contributors. “Problem solving.” https://en.wikipedia.org/wiki/Problem_solving.
2. Wikipedia contributors. “Software prototyping.” https://en.wikipedia.org/wiki/Software_prototyping.
3. Wikipedia contributors. “Usability testing.” https://en.wikipedia.org/wiki/Usability_testing.

Questions for Thought

1. The shared schedule works, but the organizer still updates it by hand. When would that be a good enough solution? What evidence would justify building a custom application?
2. All shifts are covered at the next event, but the organizer spent two extra hours calling volunteers. What improved, and what remains uncertain? What would you ask before calling the trial a success?
3. Which assumption in the chapter would you check first with only half an hour available? Explain how the answer could change your next step.
4. Think of a time someone asked you for a particular tool or feature. What difficulty might have been behind the request? How could you investigate without dismissing their proposed solution?
5. A developer says, “We already know enough. Let’s build it.” How would you decide whether another conversation is worth having or whether it is time to try something? Consider both wasted development effort and the cost of delaying useful work.